

Rime Hackathon Challenge

Build a voice-native product

Build a working product/prototype in which Rime-generated speech is essential to the experience, then try and prove that it handles one difficult voice problem under real conditions.

← Rime Hackathon Challenge ...

Speech leaves the product mostly intact, voice is not doing enough. The strongest projects will choose a problem where listening and speaking create a meaningful advantage, make Rime voices part of the primary interaction path, and demonstrate the behavior with evidence rather than claims.

The challenge

Build an end-to-end voice product for a clearly defined user and situation. Your application may use any speech-recognition, language-model, orchestration, transport, tool-use or interface stack. Rime must provide the primary spoken output.

Your project must solve at least one hard voice problem and define an acceptance test for it before the demo.

Choose one hard voice problem

- **Perceived response time:** Reduce the delay from the end of the user's turn to the first audible response. Measure the complete path a user experiences, not only one fast component.
- **Interruption and recovery:** Stop speech promptly when the user interrupts and keep the application state consistent with what the user actually heard.



Edit with the Docs app

Make tweaks, leave comments, and share

numbers, codes, addresses or domain
and before-and-after evidence.

with others to edit at the same time.

NO THANKS

GET THE APP

compatible models, voices and
get-language material.

the experience work over a phone
strained transport.

- **Expressive or persistent voice identity:** Use pacing, wording, voice choice and supported controls to create delivery that fits the character or situation and remains consistent across turns.
- **Evaluation and observability:** Build tooling that helps voice developers reproduce failures, compare implementations, measure regressions or inspect what users actually heard.

You may solve another difficult voice problem if the user impact and acceptance test are equally clear.

Suggested product directions

- **Real-time and hands-busy experiences:** Phone workflows, field operations, accessibility, commerce, public services, healthcare operations or other situations where a screen is unavailable or distracting.
- **Simulation and interactive experiences:** Training, coaching, role-play, games, storytelling or media where timing and vocal delivery affect the outcome.
- **Tools for voice builders: Testing, evaluation, pronunciation, latency instrumentation, observability, prompt experimentation or regression detection.**
- **Voice evaluation and benchmarking:** Community loves comparisons, sometimes even more than customers do. Build a fair, reproducible system comparing Rime with at least two alternative TTS systems for a clearly defined use case. Separate listening quality, text fidelity, latency, reliability and controllability. Use comparable, provider-recommended configurations; document voice selection; blind provider identities for listening tests; and distinguish model from network latency and warm from cold runs. Publish the corpus, exact configurations, generated clips, item-level results, analysis code and limitations. Report trade-offs rather than one unexplained score, and label small-sample findings as exploratory. Judges will score the fairness and usefulness of the evaluation, not whether Rime wins.

These are directions, not separate scoring tracks. A focused product with one convincingly solved voice problem is stronger than a broad assistant with many shallow features.

Rime usage requirements

- Rime-generated audio must be on the product's primary interaction path. Using Rime only for a welcome message, final confirmation or optional playback does not qualify.
- Use a production model and a compatible voice and language listed in Rime's live catalog at submission time. Test the exact combination used in the demo.
- Keep Rime and other service credentials in server-side secrets or an approved provider integration. Never commit credentials to source, documentation, screenshots, recordings or client code.
- Fallback behavior is allowed and encouraged for resilience, but it must be disclosed. The application should make the active speech provider observable, and the judged flow must demonstrate Rime as the default path.
- Rime provides text-to-speech. Your application remains responsible for user input, speech recognition, reasoning, orchestration, state, transport, tools, safety and evaluation.

What judges should see

1. A specific user, problem and reason voice is the right interface.
2. A complete working flow, not a concept deck or static interface.
3. The selected Rime model, voice, language, endpoint and transport, plus why they fit the use case.
4. A normal interaction and one deliberate stress or failure case.
5. The acceptance test, measurement or evaluation supporting the project's main voice claim.
6. Honest limitations and clear behavior when a dependency fails or input is unsupported.

Demo requirements

Submit a recorded demonstration of no more than three minutes. It must show:

- The target user and problem.
- The normal end-to-end interaction.
- The selected hard voice problem being tested.
- The result of that test, including relevant measurement or observable behavior.
- Which speech provider is active during the demonstrated flow.

Edited demos are acceptable, but the demonstrated behavior must exist in the submitted repository. Judges may ask teams to reproduce it.

Submission requirements

- A source repository that judges can inspect.
- A working demo link or recorded demo.
- A README with setup instructions, architecture, third-party services and known limitations.
- The exact Rime model ID, speaker, language, endpoint, audio format and transport used.
- A short **RIME_EVIDENCE.md** containing the project's hard voice claim, acceptance test, procedure, result and limitations.
- A repeatable command, script or fixture for the acceptance test when practical.
- An environment example containing placeholders only.
- A successful run of the organizer-provided Rime configuration and secret preflight check.

Judging criteria

Problem and necessity of voice — 25%

Does the product solve a clear problem for a defined user? Would the experience become materially worse if speech were removed?

Hard voice engineering — 25%

Did the team solve a meaningful voice-specific challenge such as latency, interruption, pronunciation, multilingual routing, telephony, delivery consistency or evaluation?

Rime integration and voice experience — 20%

Is Rime central to the working product? Do the selected model, voice, language and transport suit the situation? Is the output clear, appropriate and well integrated?

Evidence and reproducibility — 20%

Are the project's important claims supported by a transparent method, committed artifacts or a repeatable test? Does the evidence measure user-visible behavior rather than a convenient proxy?

Demo clarity — 10%

Does the demonstration clearly show the problem, working product, Rime's role, stress case and result?

Eligibility and integrity

A submission is not eligible for judging if it:

- Contains no verifiable Rime integration in the submitted code.
- Uses Rime only for incidental or optional speech.
- Provides only static screens, a concept deck or a scripted mock without a working product path.

- Omits the required demo.
- Exposes a live credential or other secret.
- Uses a model, voice or language combination that fails the event preflight and is not corrected before the deadline.

Unverified performance numbers receive no credit. Cached and uncached measurements must be labeled separately. Judges will score the shipped code and demonstrated behavior, not unsupported README claims.

Technical guidance

- Measure latency from the user's perspective: end of input to first audible response. Include speech recognition, reasoning, buffering, network, synthesis and playback where they apply.
- If the user can interrupt, stopping the audio is only half the problem. Reconcile conversation and tool state with the portion actually delivered.
- Test names, numbers, addresses, identifiers and domain vocabulary early.
- Use the current voice catalog rather than copying a stale speaker list into the application.
- Keep spoken turns concise. Long generated paragraphs increase delay and make interruption recovery harder.
- Test the exact endpoint, region, framework, model and audio format used in the final demo.
- Use synthetic or de-identified data for healthcare, finance, identity, safety and other sensitive workflows.

Starter resources

- [TTS in five minutes](#)
- [Models](#)

- [Voices and supported languages](#)
- [Live model, voice and language catalog](#)
- [Regional endpoints](#)
- [Reference voice-agent project](#)